

Predicting the Location of a Photograph

Ali Yunus Emre ÖZKÖSE
Hacettepe University
ANKARA, TURKEY

aliozkose@hacettepe.edu.tr

Tarık Ayberk YILIKOĞLU
Hacettepe University
ANKARA, TURKEY

tarikyilikoglu@hacettepe.edu.tr

Abstract

In this paper, we addressed to prediction of an image location problem. It is still a hard problem because of several kinds of other problems. We use convolutional neural networks (CNNs) to tackle this problem. We collect data from Flickr[13], create a dataset which we call Turkey15 and test with basic algorithms. After testing the dataset, we train AlexNet and ResNet-18 with Turkey15 from scratch. Since Turkey15 is very small, we use transfer learning to improve results. We use feature extracting and fine-tuning[14]. We also freeze some layers to get better accuracy.



(a) Ankara (b) İstanbul (c) Bursa

Figure 1: Images from Turkey15

1. Introduction

Although there are a lot of works on this issue and it is very popular research topic in recent years, predicting the location of an image is still a hard problem. There are various problems such that constructing features [3], viewpoint problem[4], illumination and structural modification[12] etc.. It can be used for many areas such as estimation people's perception [5]. But how can we predict the location of given image? In this work, We focus on exactly the problem of city classification.

With the development of technology and the increase of applications, people are taking photos and upload to internet much more than ever. The significant point of sharing is that a huge data has existed and it can be used for creating artificial machines as an experience. At this point, we collected images from Flickr where are taken in Turkey, cre-



Figure 2: Images from Turkey15

ated a dataset which we called Turkey15 and predict image locations where is limited to Turkey.

First of all, we tested our dataset with hand-crafted features which are Tiny images, GIST features, and Hog features, because we should know that our dataset is convenient enough to use as a dataset or not. Details in this process are explained in section 3.1.

After testing the dataset, we trained existed models which are AlexNet and ResNet-18 models with our dataset. We trained from scratch in this step and get some results and compare with training with hand-crafted features. Details and result are written in section 4.1.

Thirdly, we used transfer learning, in particular, fine-tuning and feature extracting. We trained pre-trained models which are trained with places365 and imageNet datasets. Models are AlexNet and ResNet-18 again. Details are written in section 4.2.

Finally, we froze some layers of models and trained AlexNet and ResNet-18 again. Details are written in section 4.3.

2. Related Work

Because of the popularity of this challenge, there are many kinds of proposed methods and works for predicting location. Li et al. propose to represent features with SIFT and match query image features to database image features mutually[11], but matching is only among the prioritized features. They keep informative points. In this way, they reduce computational cost. We also used hand-crafted features for testing dataset, but we use convolutional neural networks for training.

Qian et al. propose to create a visual word group for each image instead of a single bag of words [1]. They use global features to select candidates and local features [2, 9] for feature extraction. After refining features with their own visual word group (VWG) generation, VWGs are being created. The salient area is used for prediction instead of single word bu using VWGs. We both work on place images, but we use convolutional neural networks.

Isken et al. propose panorama-to-panorama matching[24]. They aggregate image features of same places on the database side and create aggregated features. They also use aggregation on query side.

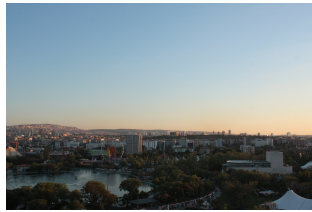
It is also significant and not easy to construct effective features for prediction. Li Fei-Fei et al. propose 5 different types of features: GPS encoding features, geographic map features, statistic features(such as TUIK for Turkey), hashtag features, visual content features [3]. They use well-known architecture AlexNet and concatenate features. The similarity with our works is using deep convolutional neural networks. we both use AlexNet, but we also use transfer learning and created our dataset.

3. Dataset

There exists a lot of datasets that related to cities [6, 7, 8] but we created our own dataset. We predict the location of images which is taken specifically in Turkey. Because time limitations; We have chosen for now 15 cities for classification: Istanbul, Ankara, Izmir, Bursa, Antalya, Adana, Konya, Gaziantep, Sanliurfa, Mersin, Trabzon, Canakkale, Rize, Adiyaman, and Nevsehir. So, we have 15 classes and we collected 150 images for each class. Our dataset consists of various viewpoints of cities: top views of the cities, inside of places, outside of places, casual places, landmarks etc..



(a) A landmark



(b) Top view of a city



(c) Inside of a landmark



(d) A casual place

We collected images from Google Street View[20] and

mostly Flickr [13] manually because labeling images would be harder. We didn't use any script or any platform which we can find images collectively.

3.1. Testing Dataset

Before explaining how we tested our dataset, we explain hand-crafted features which we used to test.

Tiny Images: We used resized Tiny images [18] as a feature. We tested our dataset with different sizes (32x32, 64x64, 128x128). It is obviously faster than real sized images because there are fewer features but we can see that using fewer features might cause smaller accuracy in *Table 1*. For example, 64x64 sized images are used as a feature and tested with logistic regression and the accuracy is 11.00 % but on the other hand, 128x128 sized images are used as a feature and tested with logistic regression and the accuracy is 13.00 %

HOG Features: HOG feature is proposed detecting humans' posture by Navneet Dalal and Bill Triggs [16], but we also used it for testing our dataset. As can be seen in the *Table 2*, we got better results than tiny images.

GIST Features: Gist feature is a low dimensional representation of the scene [17, 19]. Hence, it is used for scene recognition. We also tested our dataset with that, because of working with places. As can be seen in the *Table 3*, the accuracy increases to 41.26 %.

Image Size	Feature	Accuracy
64x64	11-NN	15.47%
64x64	13-NN	13.87%
64x64	Logistic Regression (100 iter)	11.00%
64x64	Logistic Regression (500 iter)	11.00%
128x128	1-NN	16.27%
128x128	15-NN	15.47%
128x128	Logistic Regression (100 iter)	13.00%
128x128	Logistic Regression (500 iter)	13.00%

Table 1: Tiny Images

Since we collected data to use in this project; before constructing a CNN architecture and use the dataset, we tested it by implementing KNN and logistic regression. Only 10 classes were used for the test due to some shortcomings. Various features are also used to test the dataset. Results are shown in *Table 1,2,3*



Figure 4: All cities of Turkey15

Image Size	Feature	Accuracy
32x32	21-NN	19.20%
32x32	31-NN	20.27%
32x32	Logistic Regression (100 iter)	13.00%
32x32	Logistic Regression (500 iter)	13.00%
64x64	1-NN	18.93%
64x64	11-NN	18.13 %
128x128	Logistic Regression (100 iter)	18.00%
128x128	Logistic Regression (500 iter)	18.00%

Table 2: HOG Features

Image Size	Feature	Accuracy
128x128	9-NN	27.04%
128x128	11-NN	28.89%
64x64	13-NN	28.00%
64x64	71-NN	28.53%
64x64	Logistic Regression (100 iter)	41.26%
64x64	Logistic Regression (500 iter)	41.26%
256x256	1-NN	28.15%
256x256	11-NN	25.56%

Table 3: GIST Features

As shown in *Table 1,2,3*; the results are not bad for a start. Basic models can predict locations of images, even accuracies are not high. If we want to compare, GIST features give best results of our test. The reason for higher accuracy of gist features is that gist features are already created for representation of scenes. Anyway, the results show us that collected dataset is convenient for training a model.

4. Methodology

Convolution Neural Networks (CNN) [15] are one of most popular machine learning models that have convo-

lutional layers, pooling layers and fully-connected layers. The main advantage of this model is that the model reduces the number of parameters that the model has to learn. Hence, we use CNN architectures.

We don't build a new architecture. We use the architectures that are shown to working well on images. We use AlexNet[15], the first popular CNN model, its architecture shown in *Figure 5* and ResNet[16], has a special architecture that allows the deeper models to get better results, its architecture shown in *Figure 6*.

Since Turkey15 is very small(2250 images), training from scratch using deep convolutional neural networks(CNNs) does not work well. Hence, we try to use transfer learning[21] to get better results. In particular, we use fine-tuning[14] and feature extracting.

4.1. Data Augmentation

Before explaining methods which we use, we transform datas to obtain better results and easier usage. We crop images to 224, add horizontal flips of images and finally normalize them.

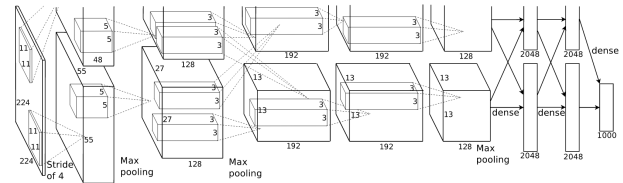


Figure 5: AlexNet

4.2. Learning from Scratch

As a baseline we train AlexNet and ResNet-18 architectures initializing random weights, that means we trained them from scratch. Even though there are deeper ResNet architectures we use ResNet-18 because it is faster to train.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹

Figure 6: ResNet

4.3. Feature Extracting

We apply feature extracting method for ResNet-18 and Alexnet models which are pre-trained with Places365[17] and ImageNet[18] datasets. In feature extracting method we remove the last layer of the pre-trained model and then we use the rest of the model as a fixed feature extractor. We add a new last layer that outputs 15 classes.

4.4. Fine-tuning

We use fine-tuning for Resnet-18 and AlexNet models. As in the feature extracting we use the models which are pre-trained with Places365 and ImageNet datasets. In fine-tuning we start with trained parameters with another dataset and we fine-tune them with training our dataset. Or we freeze some layers in the model, that means we don't fine-tune their weights. We apply only ResNet-18 model to freezing as shown in the Figure 7

	layer name	output size	18-layer
Training A	conv1	112×112	7×7, 64, stride 2
	conv2_x	56×56	3×3 max pool, stride 2
Training B	conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$
	conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$
Training C	conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$
		1×1	average pool, 15-d fc, softmax

Figure 7: Colors showing which layers frozen at trainings. For B and C also layers before them frozen.

5. Experimental Results

In this section, we explain our experiments and discuss the results. We use classification accuracy as evaluation

metric because it is more understandable and easy to interpret.

5.1. Learning From Scratch

Model	Epoch	Learning Rate	Best Validation Accuracy
AlexNet	25	0.001	10.40%
ResNet-18	25	0.001	22.40%
ResNet-18	25	0.01	15.73%
ResNet-18	25	0.005	21.07%

Table 4: Learning from Scratch

Firstly, we train AlexNet and ResNet-18 with Turkey15 as a baseline to our further experiments. Results are shown in Table 4. We don't change anything about layers or anything else. We chose the number of the epoch as 25 because of time constraints. We also try different learning rates to observe learning rate's effect and as a result, we improve the accuracy rate by up to 22.4 %. The reason behind low accuracy is using small epoch. As can be seen in Table 4, while increasing the number of epoch, the accuracy is increasing. We can not test higher epoch because of time constraints and computational cost.

Even though they are really low accuracy rates, we can say it is learning. Because Turkey15 dataset consists of 15 classes the probability of the random choice is $1/15 = 6,7\%$, we can see at the table that all of their accuracies are greater than 6,7% so we can say that all of the models we train from scratch learned something.

5.2. Feature Extracting

Model	Pretrained	Epoch	Learning Rate	Accuracy
Resnet-18	Places365	25	0.001	46.13%
Resnet-18	ImageNet	25	0.001	48.00%

Table 5: Feature Extracting

Since Turkey15 dataset is very small(2250 images) as shown in Table 4, learning from scratch using Deep Convolutional Neural Networks(CNNs) don't give good results. So we use transfer learning[20] methods. Firstly to see how suitable the features of the models which are pre-trained with Places365 and ImageNet databases we test feature extracting method. Results are shown in Table 5. It is a very big improvement compared to the learning from scratch method. The best result we get came from ResNet-18 model which is pre-trained by ImageNet database. Its accuracy is 48%. It is bigger than double of the best learning from scratch accuracy. Here we see that model pretrained with ImageNet performs better than Places365. Actually, we are expecting that Places365 will give us better results because

our problem is more similar to scene recognition. We have no idea why this happened.

5.3. Fine-tuning

Model	Pretrained	Epoch	Learning Rate	Accuracy
Resnet-18	Places365	25	0.001	55.73%
Resnet-18	ImageNet	25	0.001	52.26%
AlexNet	Places365	25	0.001	45.06%
AlexNet	ImageNet	25	0.001	46.13%

Table 6: Fine-tuning

We see the features which are learned from Places365 and ImageNet databases are working well for our problem. So we try to improve the results of transfer learning using fine-tuning method. Results of fine-tuning method shown in *Table 6*. As shown in the table *Table 6* the best results are from ResNet-18 model which is pre-trained by Places365. Its accuracy is 55,73%. It is a good improvement compared to the best result of feature extracting. We also create a graphic, shows the change in the classification accuracy in terms of the number of epochs shown in *Figure 10*. We see ResNet-18 models outperforms AlexNet models because ResNet-18 have a deeper and residual architecture.

	Model	Epochs	Learning rate	Best validation accuracy
Training A	ResNet-18	25	0.001	56.80%
Training B	ResNet-18	25	0.001	56.80%
Training C	ResNet-18	25	0.001	57.60%

Table 7: Freezing Layers

Finally, we try the improve the results of our best model. We move on with ResNet-18 model which is pre-trained with Places365 database because in previous tests it gave us the best result. We freeze some earlier layers of the network and we fine-tuned the rest of the layers. The frozen layers are shown in figure 7. The results of freezing some layers shown in *Table 7*. As shown in the table the best result is form training C. Its accuracy is 57,60%. It is a very little improvement compared to the previous result. Now we can say that our best model is this.

Since we use a new dataset, we examine the accuracies from our best model for each city and try to understand which cities are predicted well or poorly by our best model. As a result, Nevşehir is quite perfect predicted. Rize and Adiyaman cities are also well predicted compare to other cities. On the other hand, Adana and Bursa cities are predicted poorly. We are still analyzing reasons of that situation. Results are shown in *Figure 8*

We also observe training without augmentation. Result are shown in *Table 8*. The results show that when augmen-

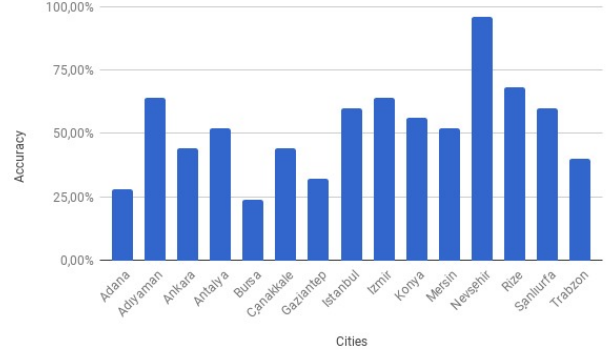


Figure 8: Accuracy results of cities from our best model

Model (Pretrained with Places365)	Epochs	Learning rate	Best train accuracy	Best validation accuracy
ResNet-18	25	0.001	0.9880%	0.5280%
ResNet-18 (freezing 3 layers)	25	0.001	0.9793%	0.5600%

Table 8: Training without augmentation

tation is not used, the overfitting problem arises. The Model memorizes dataset in training side and predicts perfect, but on the other hand, the model can't predict well as much as training side on testing side.



Figure 9: Accuracy vs epoch (Learning from scratch)

As we mentioned in section 3, Turkey15 has landmarks, casual places, different viewpoints etc.. Our best model can predict landmarks well, but if query image is casual place, it doesn't perform perfectly as much as landmarks. It is also robust to different lighting conditions.

6. Conclusion

In summary, we first create a dataset which we called Turkey15 to tackle the problem of image classification in

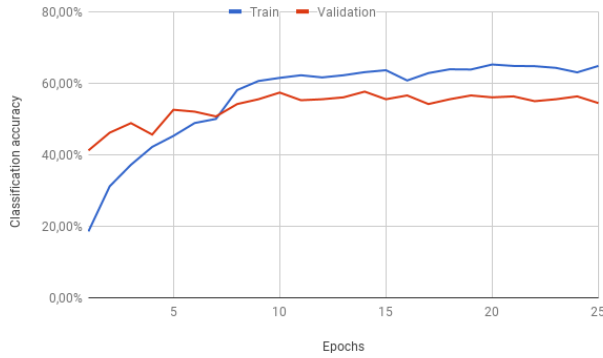


Figure 10: Accuracy vs epoch (Fine-tuning)



Figure 11: Accuracy vs epoch (Freezing layers)

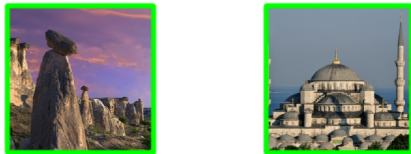


Figure 12: Landmarks



Figure 13: Casual Places

Turkey. We test Turkey15 to know if it is convenient to



Figure 14: Different lighting conditions

use or not. Then we train AlexNet and ResNet-18 from scratch with Turkey15. After that, we use transfer learning to get better results. We use feature extracting and fine-tuning with models which are pre-trained with places265 and Imagenet Dataset. We also freeze some layers to get better results. Finally, we discuss which cities are predicted well or poorly.

There are some future works to get better results. We think that aggregation[24, 25] can help improve the accuracy because if it is used on query side, it can handle occlusion, viewpoint change, and scale change problems. Furthermore, Turkey15 is too small for working with deep neural networks. A huge dataset can be created. The number of cities can be increased to 81(the number of all the cities in Turkey) and the number of images for each city can also be increased.

References

- [1] Xueming Qian, Yisi Zhao and Junwei Han "Image Location Estimation by Salient Region Matching". IEEE Transactions on Image Processing, VOL. 24 11, Nov. 2015
- [2] M. Hassabalalah, Aly Amin Abdelmgeid and Hamam A. Alshazly "Image Features Detection, Description and Matching" Springer International Publishing Switzerland 2016
- [3] Kevin Tang, Manohar Paluri, Li Fei-Fei, Rob Fergus, Lubomir Bourde "Improving Image Classification with Location Context" 10.1109/ICCV.2015.121
- [4] Qi Shan, Changchang Wu, Brian Curless "Accurate Geo-registration by Ground-to-Aerial Image Matching" Second International Conference on 3D Vision, 2014
- [5] Bolei Zhou, Liu Liu, Aude Oliva and Antonia Torralba "Recognizing City Identity via Attribute Analysis of Geo-tagged Images"
- [6] G. Toliás, Y. Avrithis, "Speeded-up, Relaxed Spatial Matching" In Proceedings of International Conference on Computer Vision (ICCV

- 2011), Barcelona, Spain, November 2011.
<http://image.ntua.gr/iva/datasets/wc/>
- [7] Y. Avrithis, Y. Kalantidis, G. Tolias, E. Spyrou "Retrieving Landmark and Non-Landmark Images from Community Photo Collections", In Proceedings of ACM Multimedia (MM 2010), Firenze, Italy, October 2010.
<http://image.ntua.gr/iva/datasets/ec50k/>
- [8] Y. Avrithis, G. Tolias, Y. Kalantidis "Feature Map Hashing: Sub-linear Indexing of Appearance and Global Geometry", In Proceedings of ACM Multimedia (MM 2010), Firenze, Italy, October 2010
<http://image.ntua.gr/iva/datasets/ec1m/>
- [9] Jianxin Wu, and James M. Rehg "Centrist: a visual descriptor for scene categorization" IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 8, pp. 1489-1501, Aug. 2011
- [10] Liu, B., Yuan, Q., Cong, G. and Xu, D., *Where your photo is taken: Geolocation prediction for social images* J Assn Inf Sci Tec, 65: 1232–1243. doi:10.1002/asi.23050
- [11] Li Y., Snavely N., Huttenlocher D.P. (2010) *Location Recognition Using Prioritized Feature Matching* In: Daniilidis K., Maragos P., Paragios N. (eds) Computer Vision – ECCV 2010. ECCV 2010. Lecture Notes in Computer Science, vol 6312. Springer, Berlin, Heidelberg
- [12] Hyo Jin Kim, Enrique Dunn, Jan-Michael Frahm "Learned Contextual Feature Reweighting for Image Geo-Localization" Computer Vision and Pattern Recognition (CVPR) 2017 IEEE Conference on, pp. 3251-3260, 2017, ISSN 1063-6919.
- [13] Flickr
<https://www.flickr.com>
- [14] Becherer, N., Pecarina, J., Nykl, S. et al. "Improving optimization of convolutional neural networks through parameter fine-tuning" Neural Comput & Applic (2017). <https://doi.org/10.1007/s00521-017-3285-0>
- [15] Alex Krizhevsky and Sutskever, Ilya and Hinton, Geoffrey E "ImageNet Classification with Deep Convolutional Neural Networks", Curran Associates, Inc., 2012
- [16] N. Dalal and B. Triggs "Histograms of oriented gradients for human detection", 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), San Diego, CA, USA, 2005, pp. 886-893 vol. 1.
- [17] N. Dalal and B. Triggs "Histograms of oriented gradients for human detection", 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), San Diego, CA, USA, 2005, pp. 886-893 vol. 1.
- [18] A. Torralba, R.Fergus and W. T. Freeman "Tiny Images", 2007
- [19] Matthijs Douze, Hervé Jégou, Harsimrat Sandhawalia, Laurent Amsaleg, and Cordelia Schmid. "Evaluation of GIST descriptors for web-scale image search", In Proceedings of the ACM International Conference on Image and Video Retrieval (CIVR '09). ACM, New York, NY, USA, , Article 19 , 8 pages. 2009
- [20] Google Street View
<https://www.instantstreetview.com>
- [21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun "Deep Residual Learning for Image Recognition" CVPR 2016: 770-778
- [22] B. Zhou, A. Lapedriza, A. Khosla, A. Oliva and A. Torralba "Places: A 10 million Image Database for Scene Recognition" IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PP, no. 99, pp. 1-1
- [23] Olga Russakovsky*, Jia Deng*, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg and Li Fei-Fei *ImageNet Large Scale Visual Recognition Challenge* IJCV, 2015
["http://www.image-net.org"](http://www.image-net.org)
- [24] Ahmet Iscen, Giorgos Tolias, Yannis Avrithis, Teddy Furon and Ondřej Chum *Panorama to panorama matching for location recognition* ACM International Conference on Multimedia Retrieval 2017 (ICMR)
- [25] R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, J. Sivic *NetVLAD: CNN architecture for weakly supervised place recognition* IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017